



ROBIT
ROBOT SPORT GAME TEAM

C Language 기초 - 6일차

Pointer, 자료구조

목차



ROBIT
ROBOT SPORT GAME TEAM

Part 1 포인터와 구조체

Part 2 자료구조 기초

Part 3 파일 입출력

Part 4 선행 처리

Part 5 과제 모음



ROBIT
ROBOT SPORT GAME TEAM

포인터

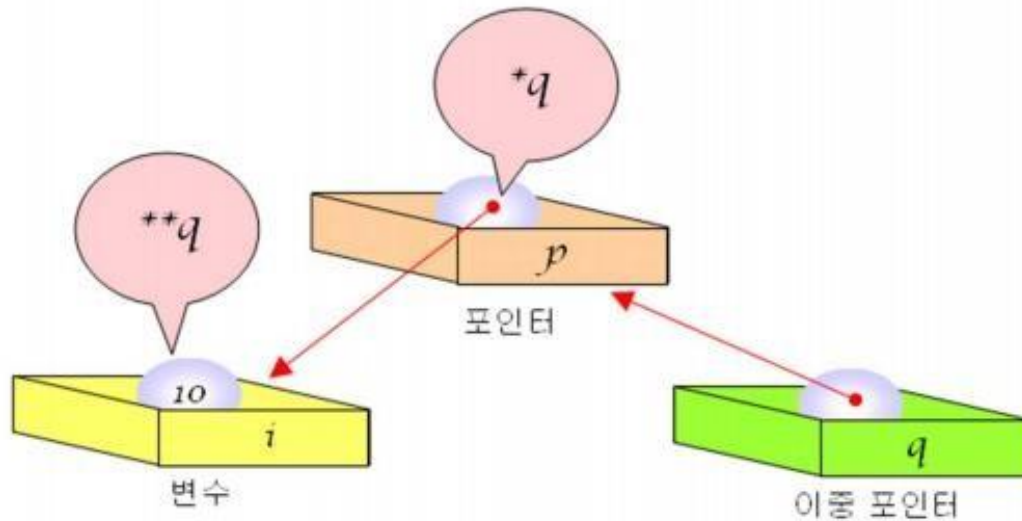
이중 포인터



ROBIT
ROBOT SPORT GAME TEAM

- 이중 포인터(double pointer): 포인터를 가리키는 포인터

```
int i = 100;           // i는 int형 변수  
int *p = &i;           // p는 i를 가리키는 포인터  
int **q = &p;          // q는 포인터 p를 가리키는 이중 포인터
```



이중 포인터



ROBIT
ROBOT SPORT GAME TEAM

```
// 이중 포인터 프로그램
```

```
#include <stdio.h>
```

```
void set_proverb(char **q);
```

```
int main(void)
```

```
{
```

```
    char *s = NULL;
```

```
    set_proverb(&s);
```

```
    printf("selected proverb = %s\n", s);
```

```
    return 0;
```

```
}
```

```
void set_proverb(char **q)
```

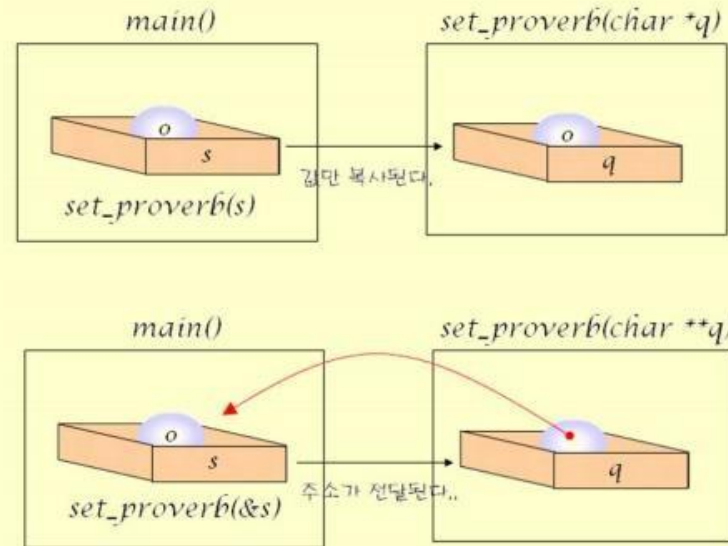
```
{
```

```
    static char *str1="A friend in need is a friend indeed";
```

```
    static char *str2="A little knowledge is a dangerous thing";
```

```
    *q = str1;
```

```
}
```



selected proverb = A friend in need is a friend indeed

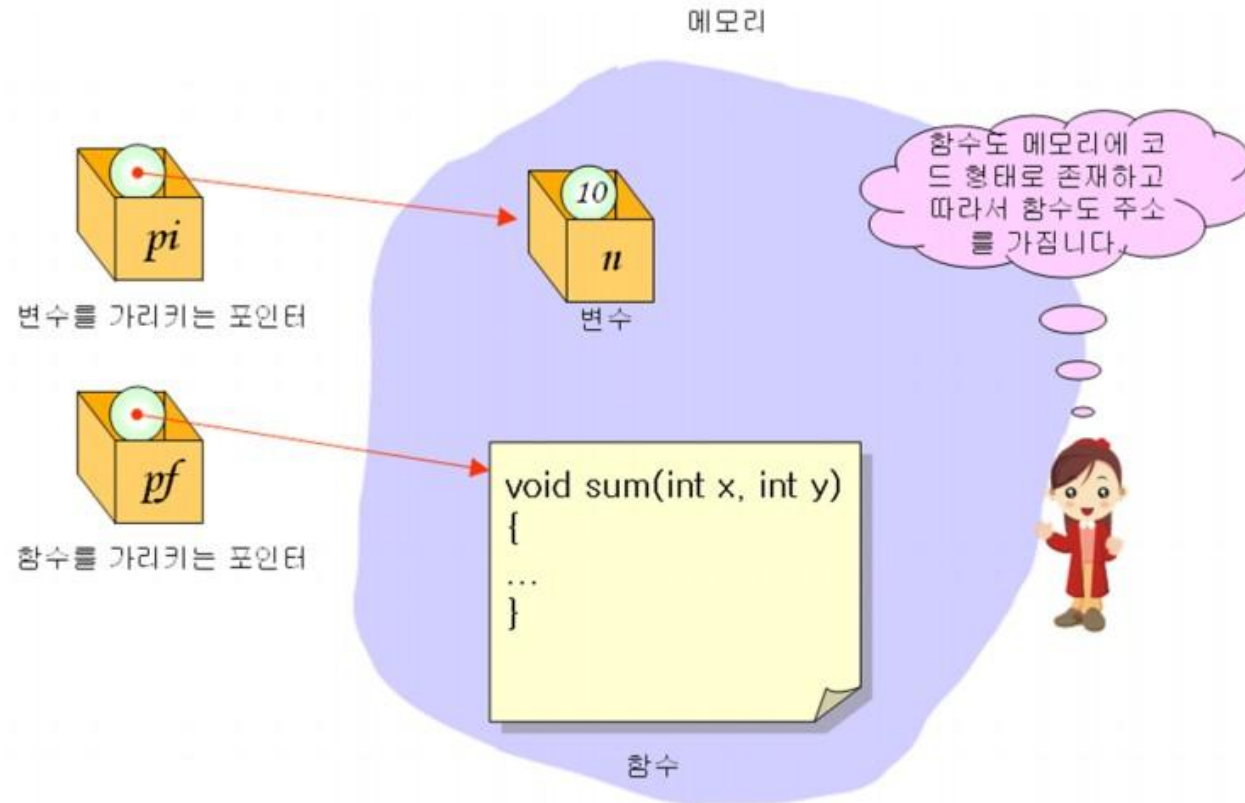
함수 포인터



ROBIT
ROBOT SPORT GAME TEAM

- 함수 포인터(function pointer): 함수를 가리키는 포인터

반환형 (*함수포인터이름) (매개변수1, 매개변수2, ...);



함수 포인터



ROBIT
ROBOT SPORT GAME TEAM

```
// 함수 포인터
#include <stdio.h>

// 함수 원형 정의
int add(int, int);
int sub(int, int);

int main(void)
{
    int result;
    int (*pf)(int, int);           // 함수 포인터 정의

    pf = add;                     // 함수 포인터에 함수 add()의 주소 대입
    result = pf(10, 20);          // 함수 포인터를 통한 함수 add() 호출
    printf("10+20은 %d\n", result);

    pf = sub;                     // 함수 포인터에 함수 sub()의 주소 대입
    result = pf(10, 20);          // 함수 포인터를 통한 함수 sub() 호출
    printf("10-20은 %d\n", result);

    return 0;
}
```

```
int add(int x, int y)
{
    return x+y;
}

int sub(int x, int y)
{
    return x-y;
}
```

10+20은 30
10-20은 -10

함수 포인터



ROBIT
ROBOT SPORT GAME TEAM

- 함수 포인터도 인수로 전달이 가능하다.



```
int main(void)
{
    ...
    sub(f);
    ...
}
```

```
void f()
{
    ...
}
```

```
int sub( void (*pf)() )
{
    ...
    pf();
    ...
}
```




ROBIT
ROBOT SPORT GAME TEAM

구조체 포인터

구조체 포인터



ROBIT
ROBOT SPORT GAME TEAM

```
#include <stdio.h>
|
typedef struct _Student
{
    int number;
    char name[10];
    double grade;
}Student;
```

```
void main(void)
{
```

```
    Student *p;
    Student s = {44, "kim", 4.5};

    p = &s;
```

```
int num= 10;
int * iptr = &num;
```

일반적인 포인터 변수의 선언 및 연산과 동일
포인터 변수 p가 구조체 변수 s를 가리킴

```
printf("학번=%d 이름=%s 학점=%f \n", s.number, s.name, s.grade);
printf("학번=%d 이름=%s 학점=%f \n", (*p).number, (*p).name, (*p).grade);
```

```
}
```

```
학번=44 이름=kim 학점=4.500000
학번=44 이름=kim 학점=4.500000
계속하려면 아무 키나 누르십시오 . . .
```

구조체 포인터



ROBIT
ROBOT SPORT GAME TEAM

연산자 -> : 구조체 포인터로 구조체 멤버를 참조

```
#include <stdio.h>
```

```
typedef struct _Student  
{  
    int number;  
    char name[10];  
    double grade;  
}Student;
```

```
void main(void)
```

```
{  
    Student *p;  
    Student s = {44, "kim", 4.5};
```

```
    p = &s;
```

```
    printf("학번=%d 이름=%s 학점=%f \n", p->number, p->name, p->grade);
```

```
}
```

학번=44 이름=kim 학점=4.500000
계속하려면 아무 키나 누르십시오 . . .

***연산과 .연산 = -> 연산**

포인터를 멤버로 가지는 구조체



ROBIT
ROBOT SPORT GAME TEAM

```
#include <stdio.h>

typedef struct _Date
{
    int month;
    int day;
    int year;
}Date;

typedef struct _Student
{
    int number;
    char name[10];
    double grade;
    Date *mdy;
}Student;

void main(void)
{
    Student s = {44, "kim", 4.5};
    Date d = {6, 26, 1995};

    s.mdy = &d;

    printf("학번 : %d, 이름 : %s, 학점 : %lf\n", s.number, s.name, s.grade);
    printf("생년월일: %d년 %d월 %d일\n", s.mdy->year, s.mdy->month, s.mdy->day);
}
```

학번 : 44, 이름 : kim, 학점 : 4.500000
생년월일: 1995년 6월 26일
계속하려면 아무 키나 누르십시오 . . .

포인터를 멤버로 가지는 구조체



ROBIT
ROBOT SPORT GAME TEAM

```
#include <stdio.h>

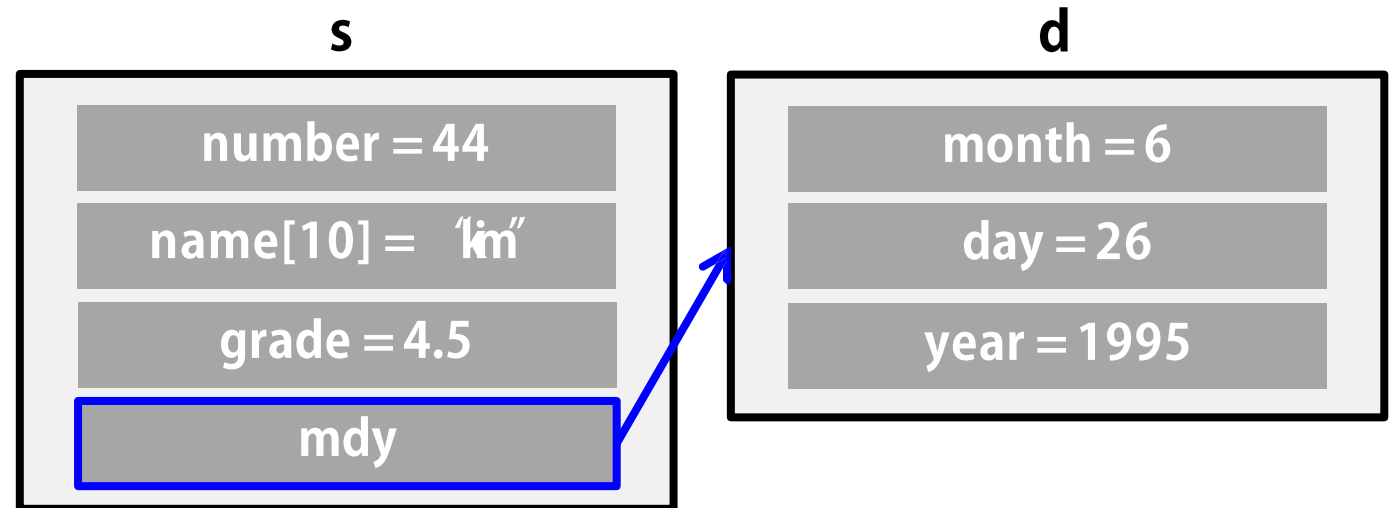
typedef struct _Date
{
    int month;
    int day;
    int year;
}Date;

typedef struct _Student
{
    int number;
    char name[10];
    double grade;
    Date *mdy;
}Student;

void main(void)
{
    Student s = {44,"kim",4.5};
    Date d = {6,26,1995};

    s.mdy = &d;

    printf("학번 : %d, 이름 : %s, 학점 : %lf \n", s.number,s.name,s.grade);
    printf("생년월일: %d년 %d월 %d일\n", s.mdy->year, s.mdy->month, s.mdy->day);
}
```



구조체 배열 동적할당



ROBIT
ROBOT SPORT GAME TEAM

구조체도 malloc을 통해 동적할당을 할 수 있다.

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

//학생 정보 구조체
typedef struct _Student
{
    int number;
    char name[20];
}Student;

void ADD(Student* list, int number)
{
    (*(list + number)).number = number;
    printf("이름을 입력하시오 : ");
    scanf("%s", (*(list + number)).name);

    //list[number].number = number;
    //printf("이름을 입력하시오 : ");
    //scanf("%s", list[number].name);
}
```

```
int main(void)
{
    Student* list;

    int student_num = 2;

    //배열 동적 할당
    list = (Student*)malloc(sizeof(Student) * student_num);

    ADD(list, 0);
    ADD(list, 1);

    for (int i = 0; i < student_num; i++)
    {
        printf("-----#n");
        printf("이름 : %s#n", list[i].name);
        printf("번호 : %d#n#n", list[i].number);
    }
}
```

realloc



ROBIT
ROBOT SPORT GAME TEAM

malloc을 통해 할당된 메모리의 공간을 더 늘리거나 줄이기 위해 사용한다.

realloc(pointer, size);

```
int main(void)
{
    Student* list;

    int student_num = 2;

    //배열 동적 할당
    list = (Student*)malloc(sizeof(Student) * student_num);
    ADD(list, 0);
    ADD(list, 1);

    for (int i = 0; i < student_num; i++)
    {
        printf("-----\n");
        printf("이름 : %s\n", list[i].name);
        printf("번호 : %d\n\n", list[i].number);
    }

    student_num = 3;
    list = (Student*)realloc(list, sizeof(Student) * (student_num));
    ADD(list, 2);

    for (int i = 0; i < student_num; i++)
    {
        printf("-----\n");
        printf("이름 : %s\n", list[i].name);
        printf("번호 : %d\n\n", list[i].number);
    }

    //메모리 해제
    free(list);
    return 0;
}
```



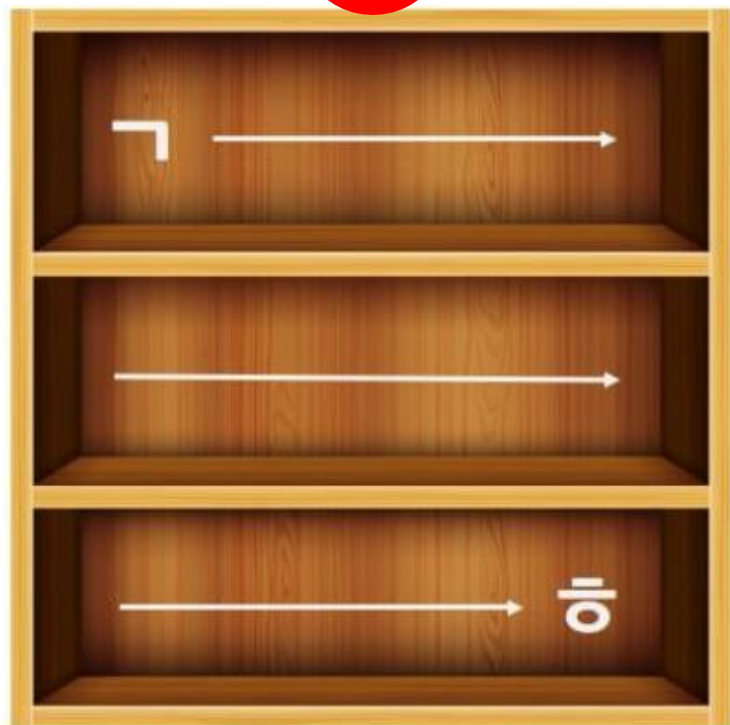

ROBIT
ROBOT SPORT GAME TEAM

자료구조



ROBIT
ROBOT SPORT GAME TEAM

1



2



자료구조



ROBIT
ROBOT SPORT GAME TEAM

- 프로그램 :
 - 데이터를 표현하고 그렇게 표현된 데이터를 처리하는 것
 - 데이터를 표현하고 그렇게 표현된 데이터를 처리하는 역할을 수행하는 일련의 명령어들의 집합
- 자료 구조 :
 - 데이터(자료)의 집합
 - 데이터를 저장 및 정렬하는 방식
 - 자료 구조의 목적 → 데이터를 효율적으로 처리하는 것

자료구조의 이용



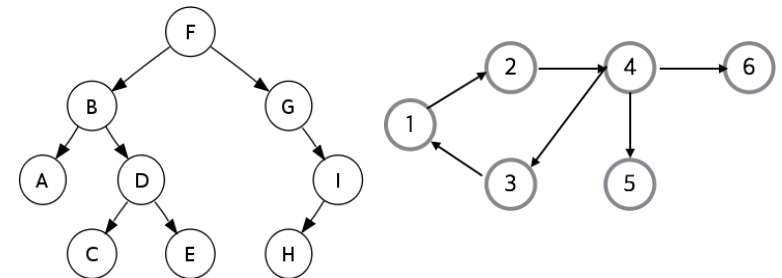
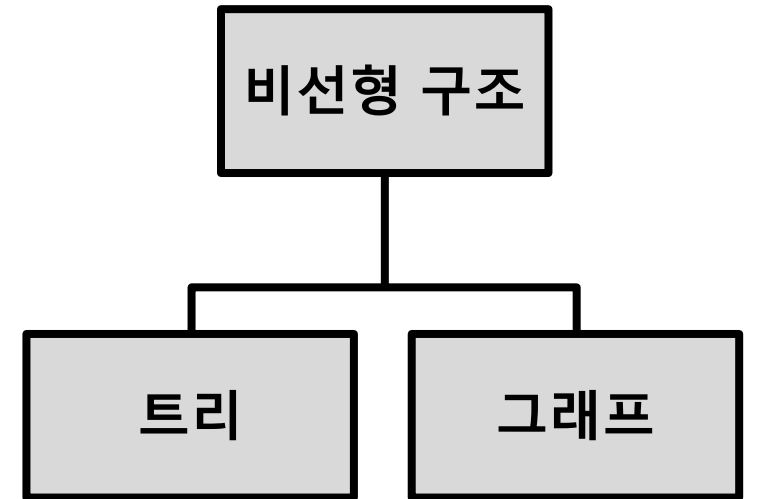
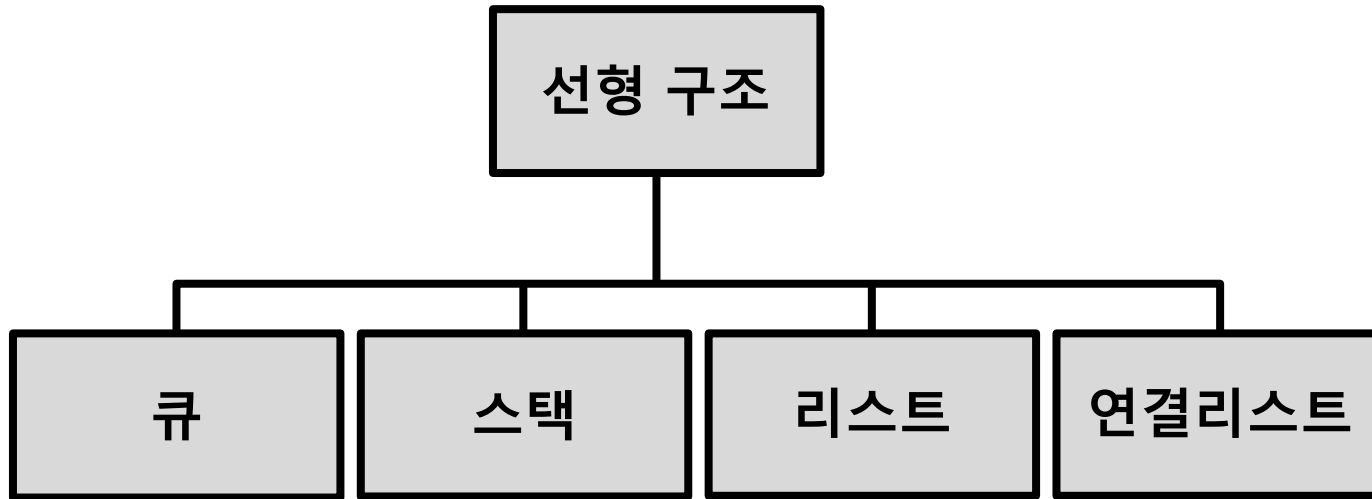
ROBIT
ROBOT SPORT GAME TEAM

- **정렬(sort)**
: 기억장치 내의 자료를 일정한 순서에 의해 나열하기
- **검색(Search)**
: 기억장치 내의 자료를 찾기
- **파일 편성**
: 자료를 특정한 구조로 기억 매체에 저장하기
- **인덱스**
: 특정 자료를 빠르게 찾기 위한 색인표 지정

자료구조의 분류



ROBIT
ROBOT SPORT GAME TEAM

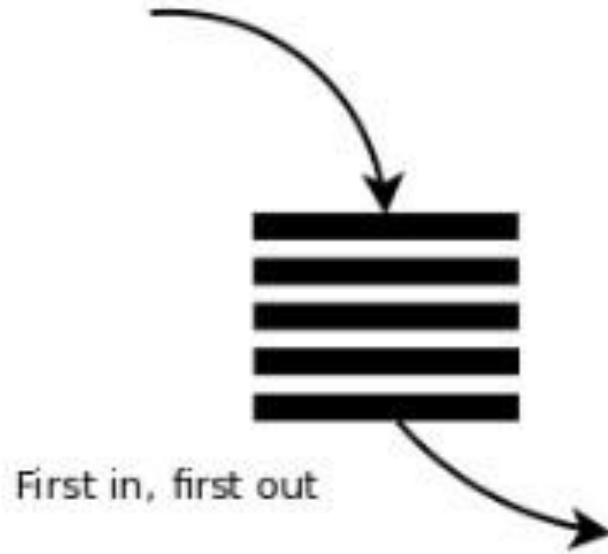


큐, 스택



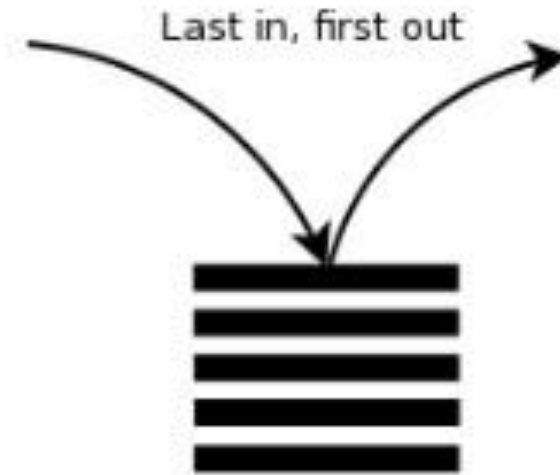
ROBIT
ROBOT SPORT GAME TEAM

Queue:



- 선입선출(FIFO : First In First Out)

Stack:



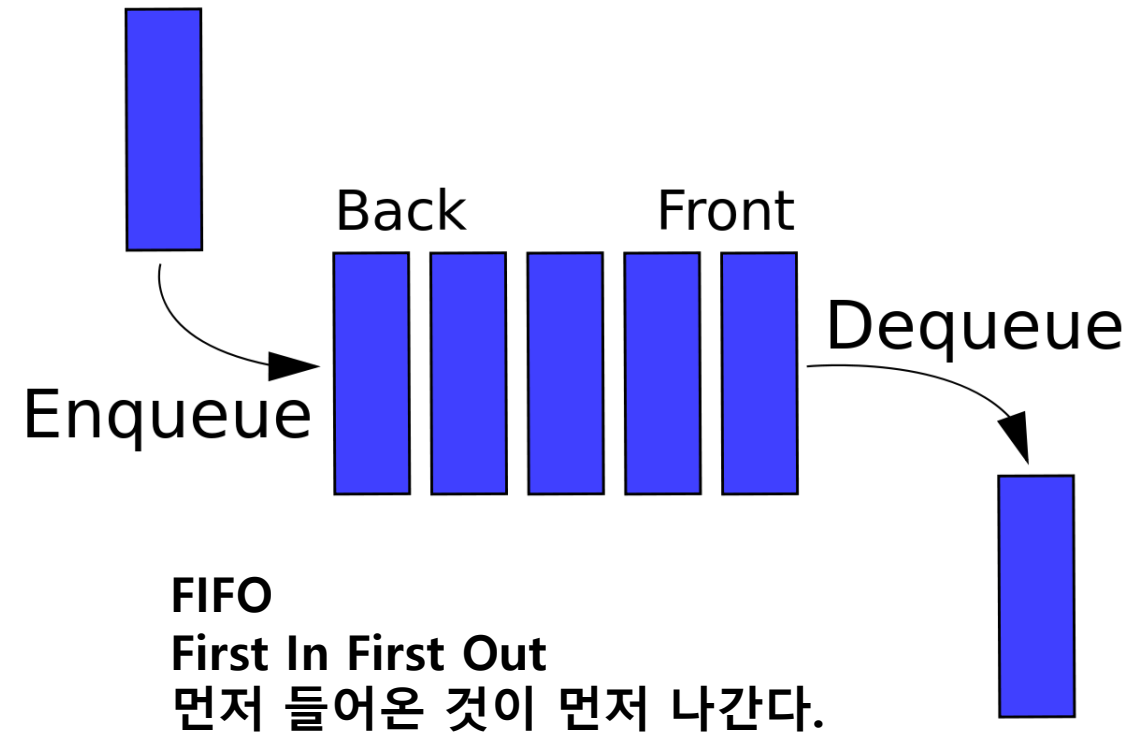
- 후입선출(LIFO : Last In First Out)

Queue



ROBIT
ROBOT SPORT GAME TEAM

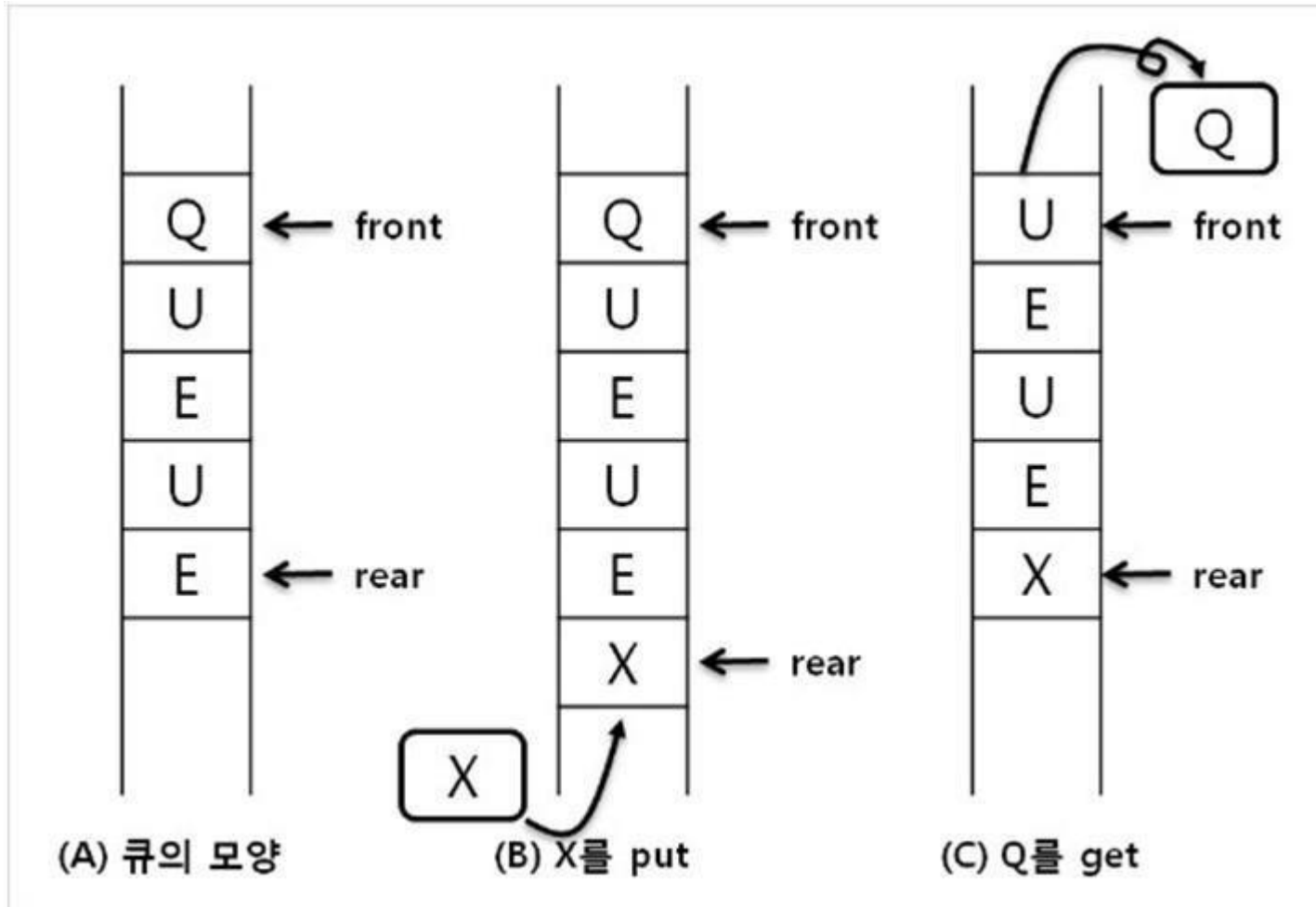
- 데이터 삭제시, 재정렬이 필요없다.
- 사용 운영체제(OS)에서 프로세스 스케줄링 방식을 구현하기 위해 사용된다.
- 창구 업무나 택시 정거장처럼 서비스 순서를 기다리는 등의 대기 행렬의 처리에도 사용된다.



Queue



ROBIT
ROBOT SPORT GAME TEAM



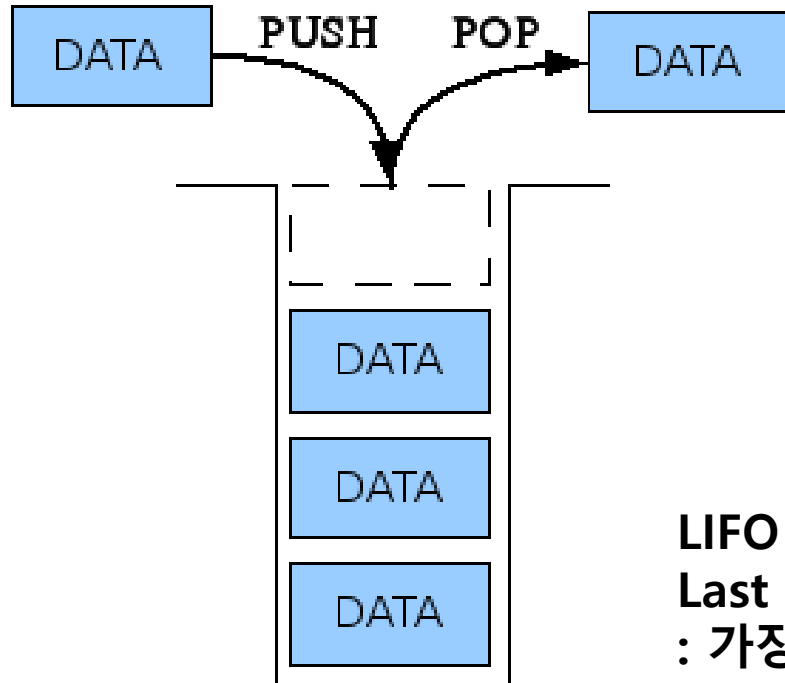
- Enqueue(=put=push) : queue의 rear(back)에 새로운 원소 삽입
- Dequeue(=get=pop) : queue의 front에 있는 원소를 queue로부터 삭제하고 반환하는 연산

Stack



ROBIT
ROBOT SPORT GAME TEAM

- 구현이 단순하여 쉽고 데이터의 저장 및 검색 속도가 빠르다.
- 저장될 수 있는 데이터의 최대개수를 미리 정해야한다. 즉, 저장공간의 낭비가 발생한다.
- 함수 호출의 순서 제어 등 프로세스 실행구조의 기본으로 사용되는 자료구조이다.



LIFO

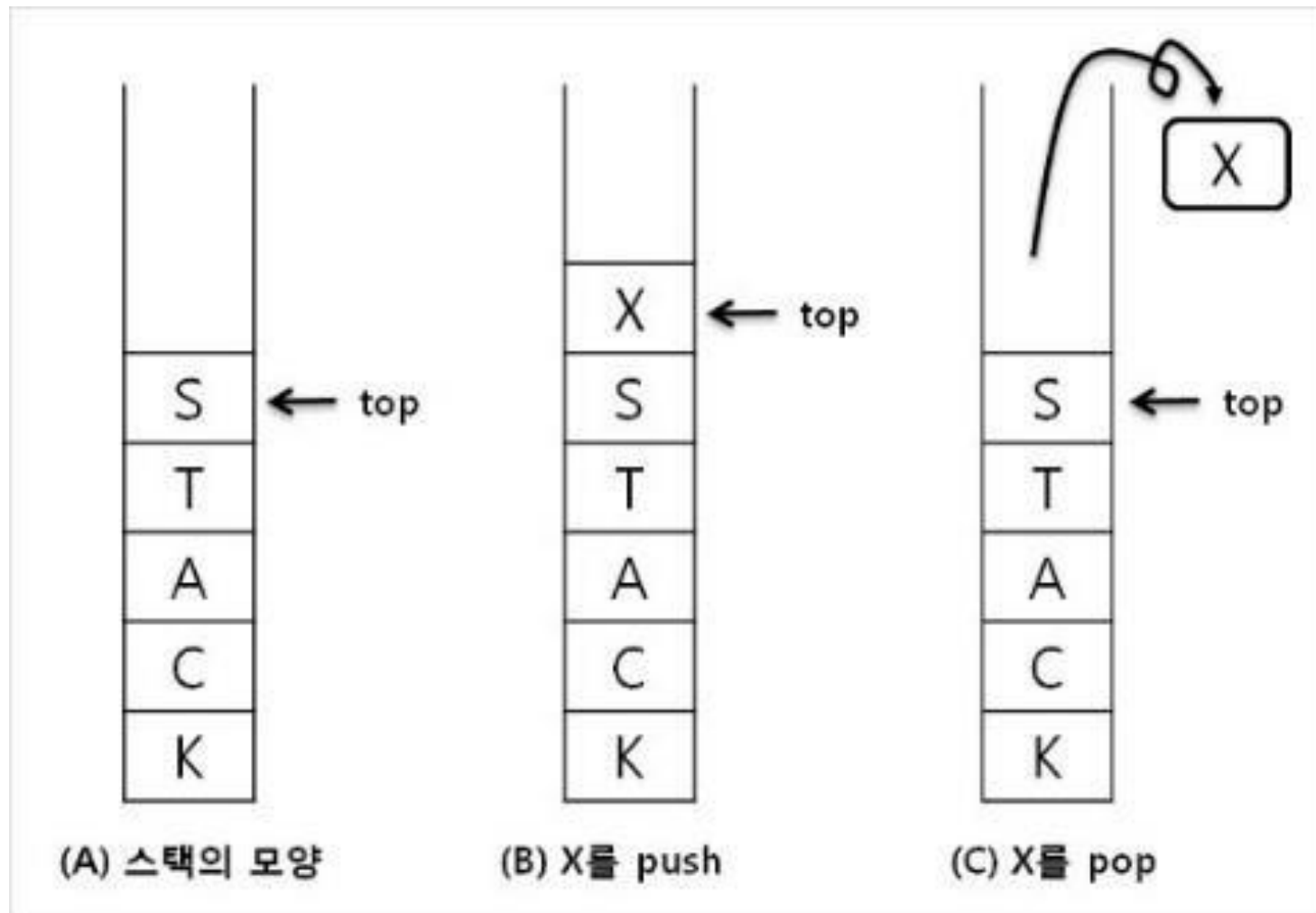
Last In First Out

: 가장 최근에 들어온 데이터가 가장 먼저 나가게 됨.

Stack



ROBIT
ROBOT SPORT GAME TEAM





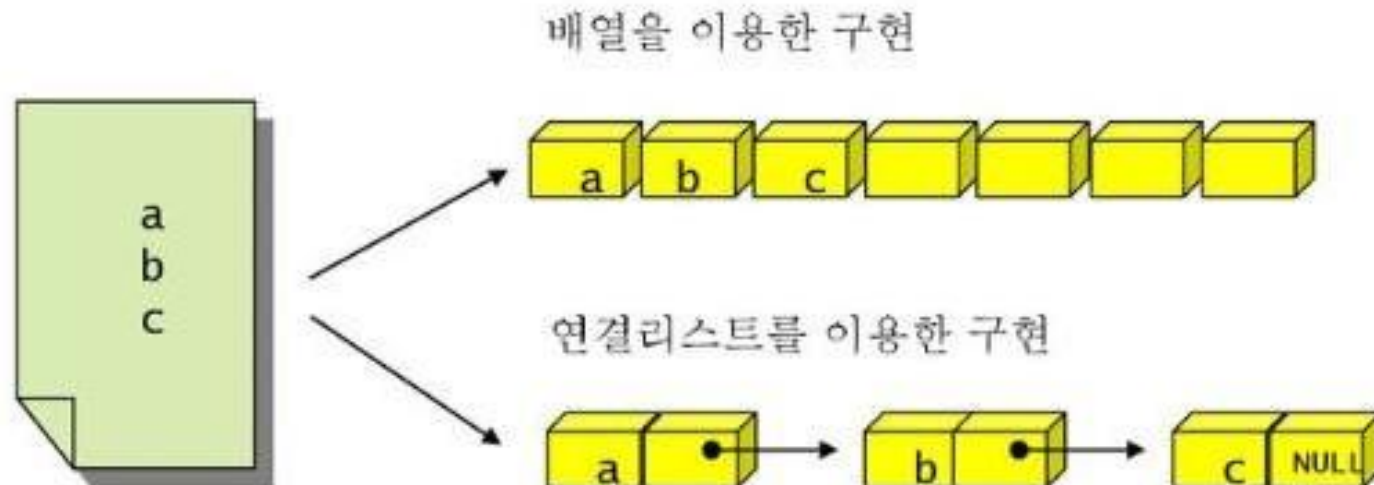
ROBIT
ROBOT SPORT GAME TEAM

Linked list

Linked list



ROBIT
ROBOT SPORT GAME TEAM



리스트(ex-배열) :

메모리 공간에서 연속.

빠른 속도가 보장되지만 메모리가 연속이어야 하기 때문에 삽입과 삭제가 번거로움

연결리스트 :

가변적인 배열.

크기가 제한되지 않고 메모리 상에 연속이지 않아도 되기 때문에
중간에 데이터를 추가하거나 삭제하기 편리함.

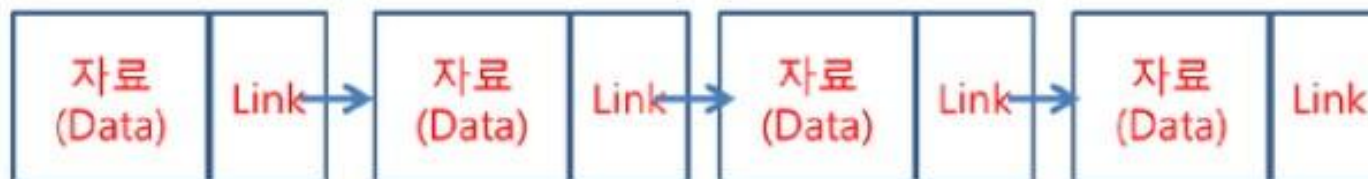
구현이 복잡하고 임의의 항목을 추출할 때 많은 시간 소요

Linked list

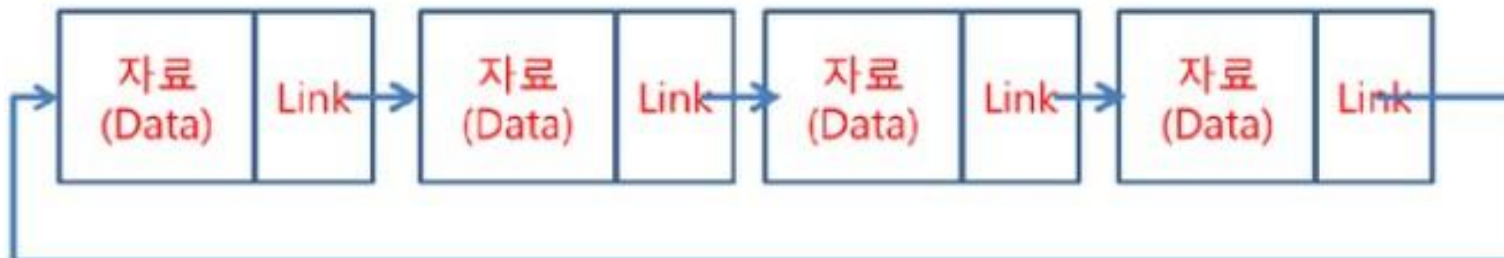


ROBIT
ROBOT SPORT GAME TEAM

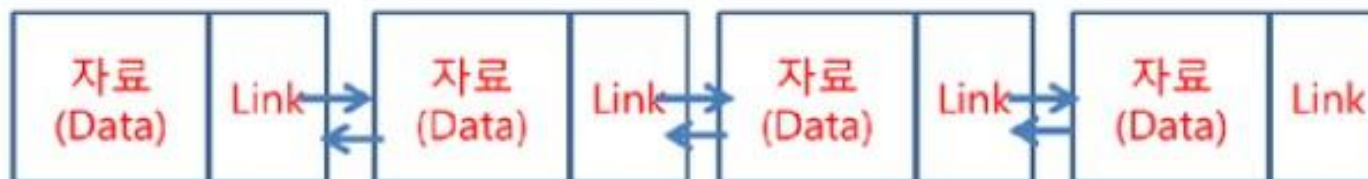
단순 연결 리스트



원형 연결 리스트



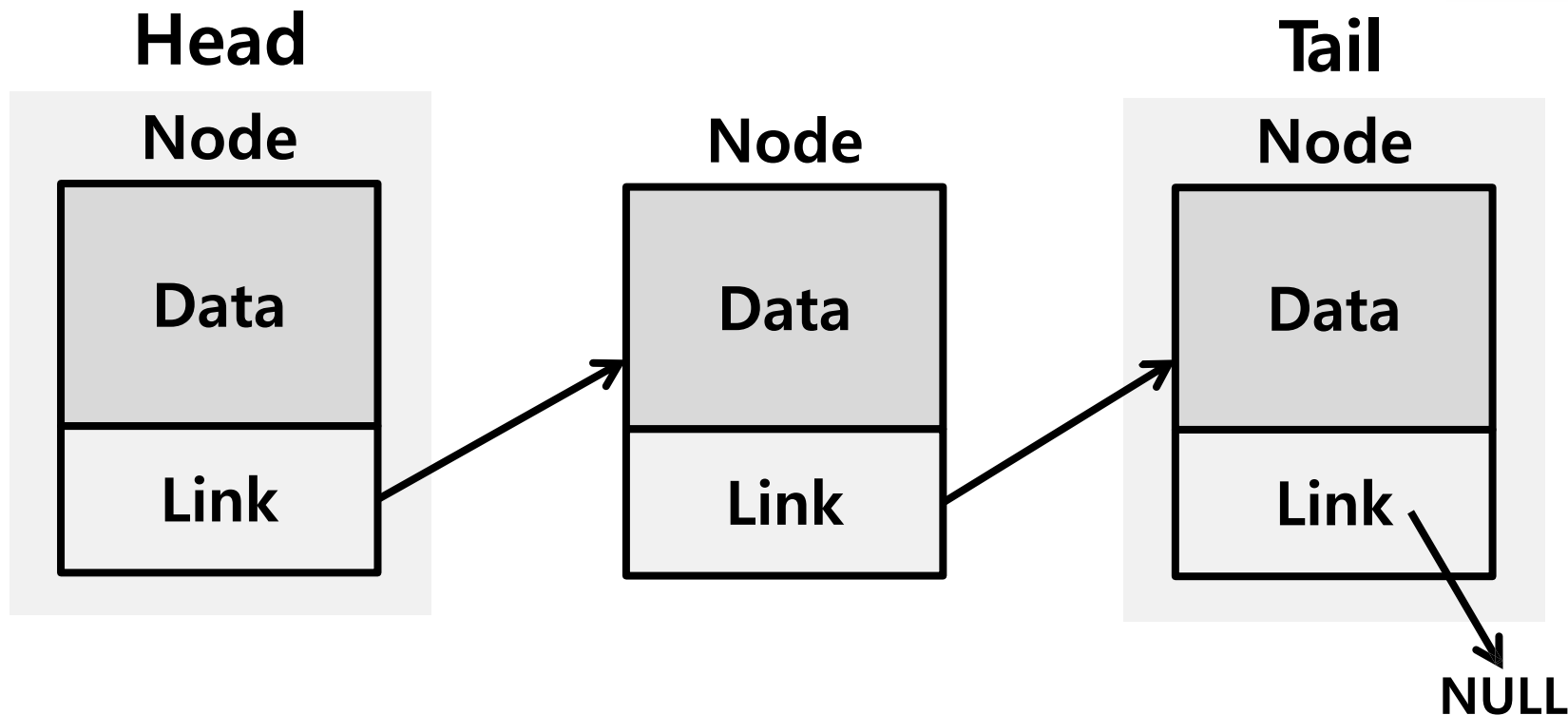
이중 연결 리스트



Linked list



ROBIT
ROBOT SPORT GAME TEAM



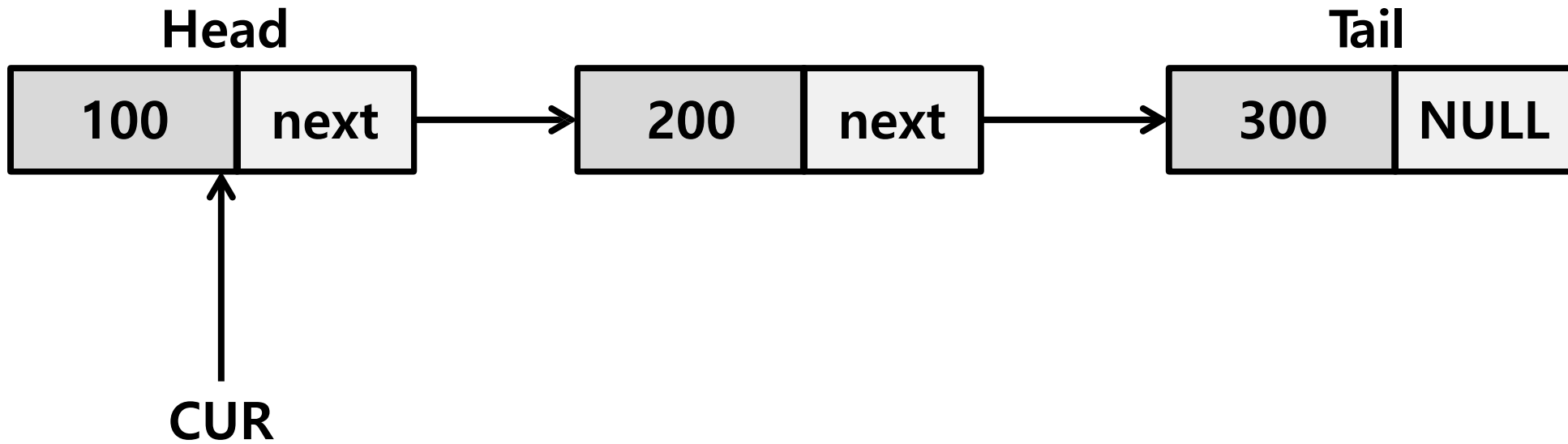
```
typedef struct _Node {  
    int data;  
    struct _Node * next;  
}Node;  
  
typedef struct _LinkedList {  
    Node * head;  
    Node * tail;  
    Node * cur;  
    int size;  
}LinkedList;
```

- 연결 리스트 = Node들의 집합
- Node : 데이터의 저장단위로, 자료(Data)와 포인터(Link, 노드의 위치정보)가 한 쌍.
 - Data field : 저장하고자 하는 데이터가 들어감
 - Link field : 다른 노드를 가리키는 포인터가 저장되고 이를 이용하여 다른 노드에 접근 가능

Search



ROBIT
ROBOT SPORT GAME TEAM



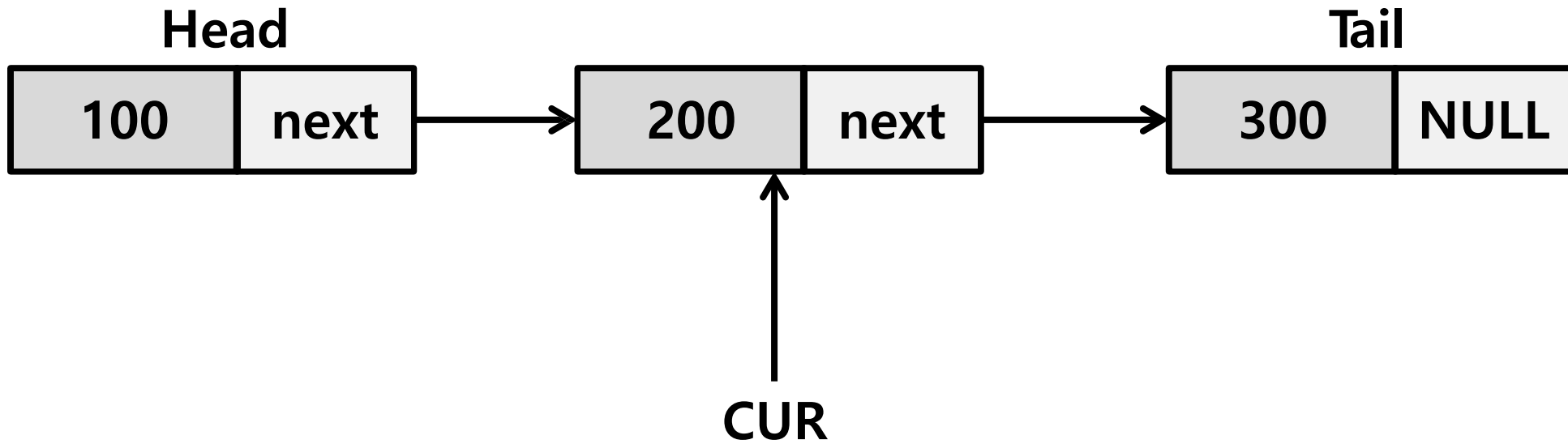
`CUR = CUR->next;`

- 탐색 방향이 일편적
- 순차적으로 노드를 세어나가 원하는 요소에 접근해야 한다는 한계점

Search



ROBIT
ROBOT SPORT GAME TEAM

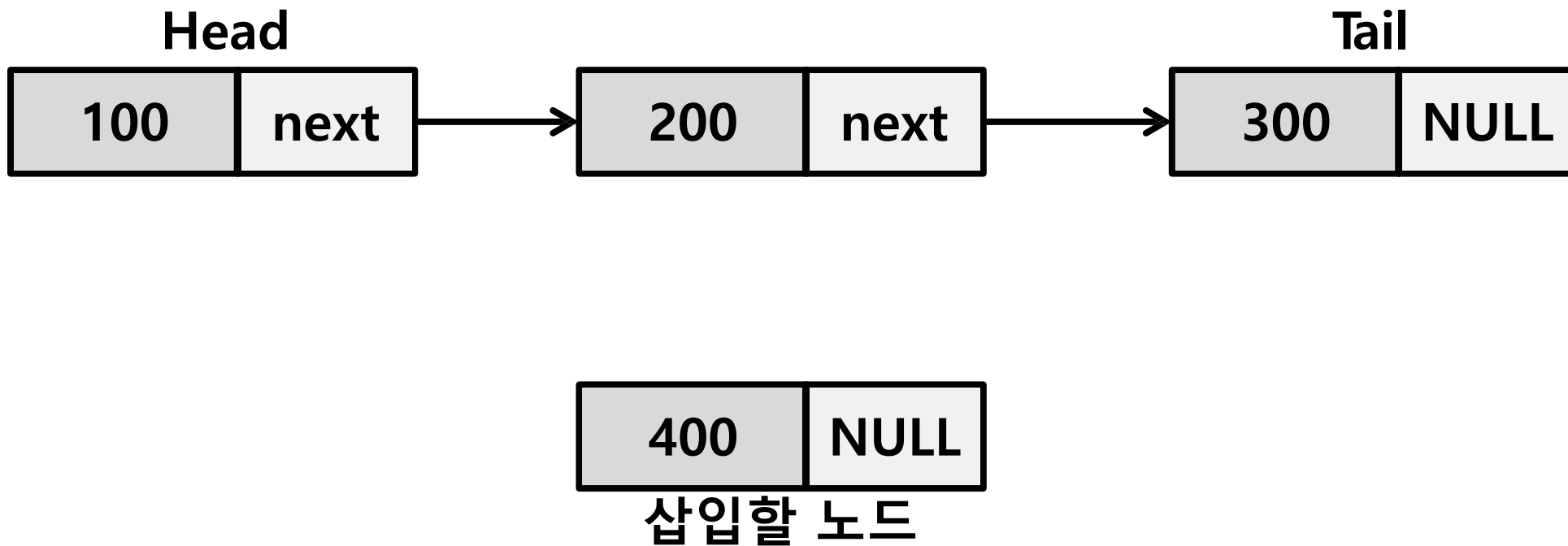


CUR = CUR->next;

Insert



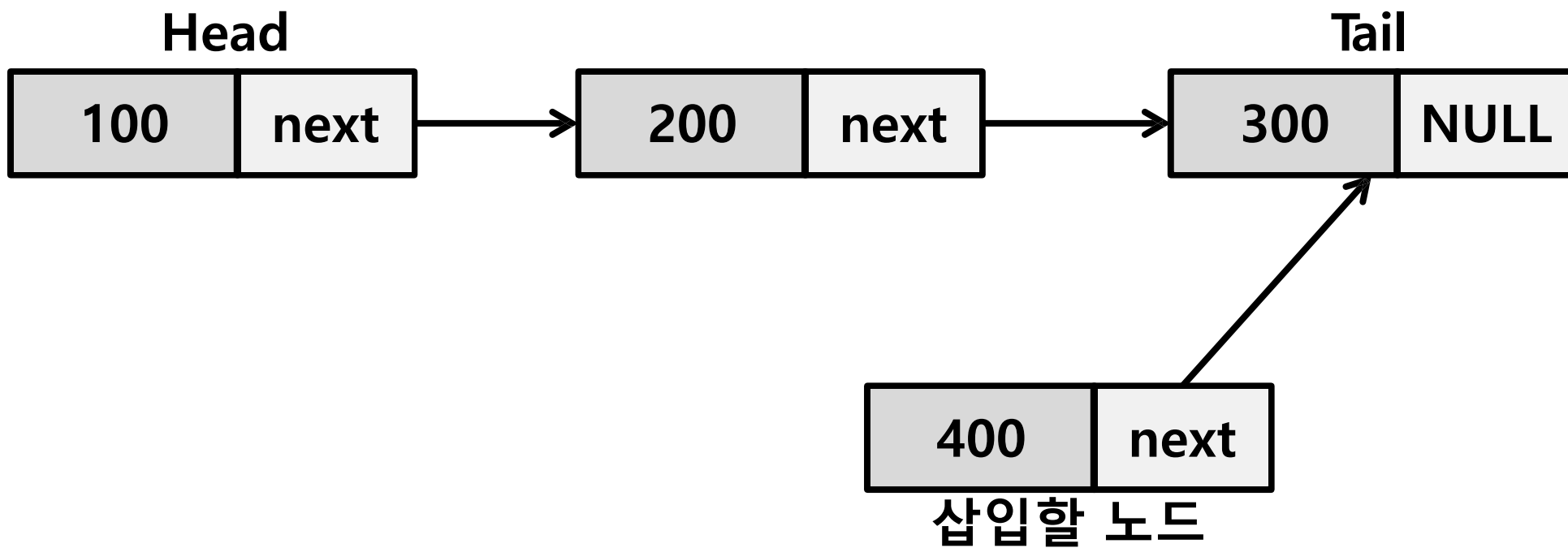
ROBIT
ROBOT SPORT GAME TEAM



Insert



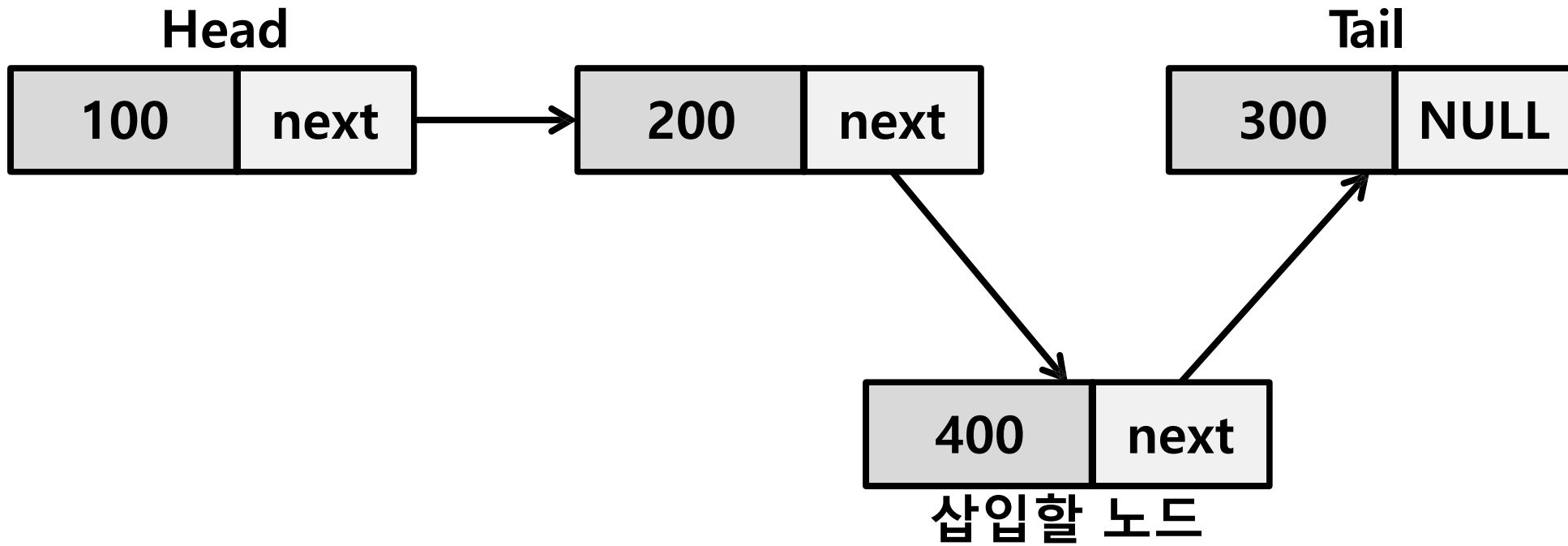
ROBIT
ROBOT SPORT GAME TEAM



Insert



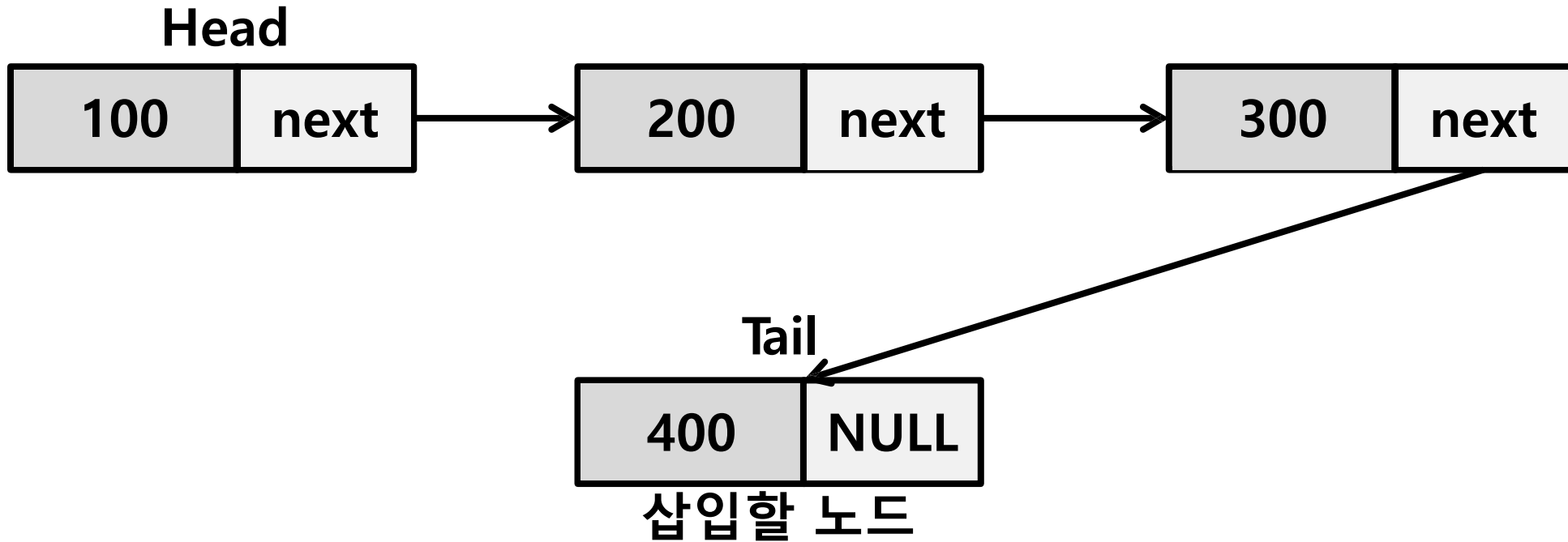
ROBIT
ROBOT SPORT GAME TEAM



Insert_back



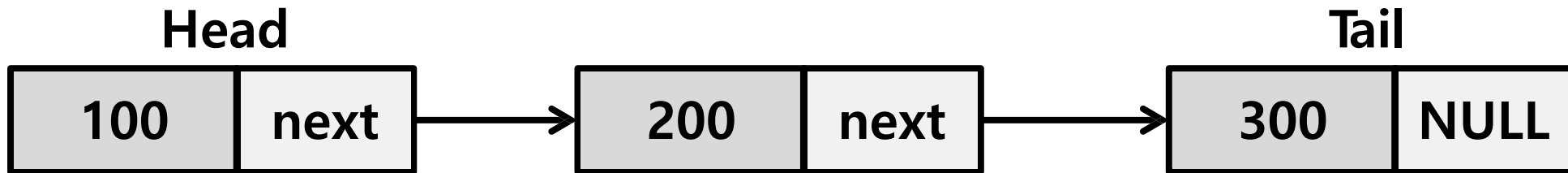
ROBIT
ROBOT SPORT GAME TEAM



Delete



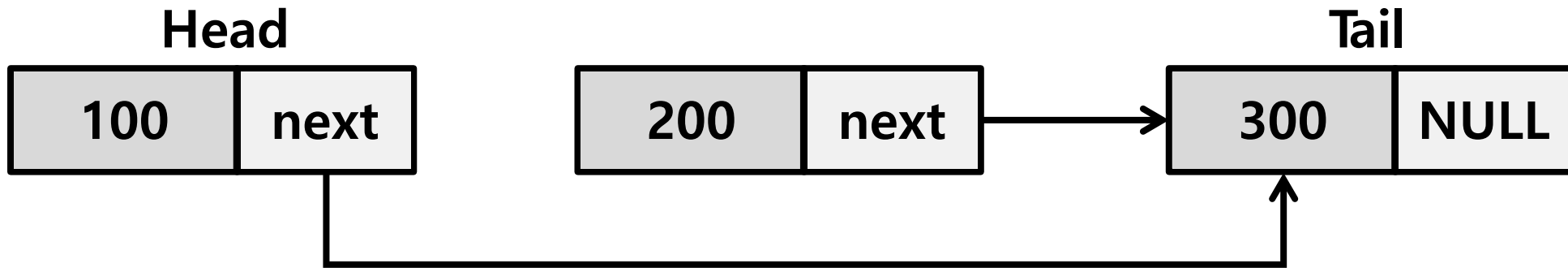
ROBIT
ROBOT SPORT GAME TEAM



Delete



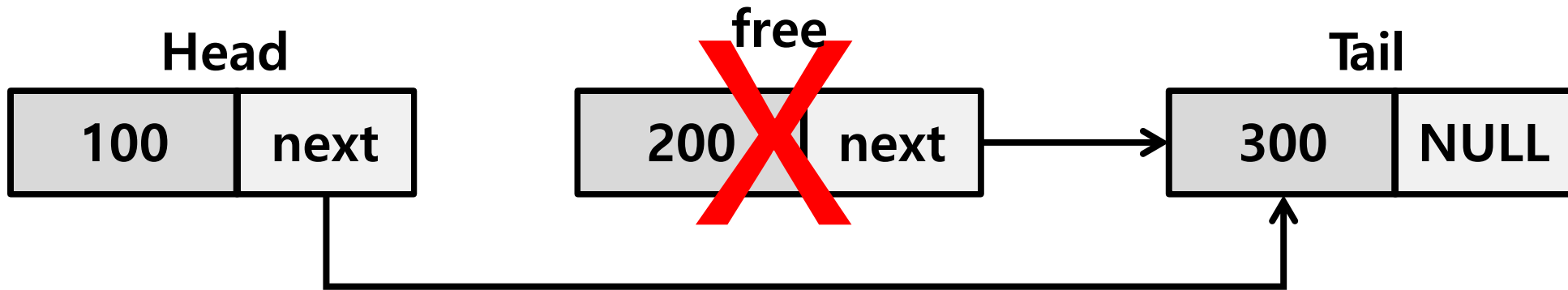
ROBIT
ROBOT SPORT GAME TEAM



Delete



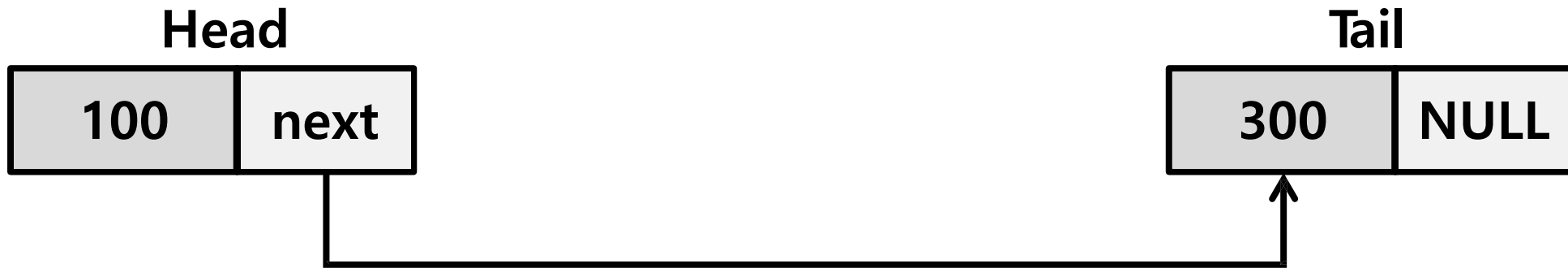
ROBIT
ROBOT SPORT GAME TEAM



Delete



ROBIT
ROBOT SPORT GAME TEAM





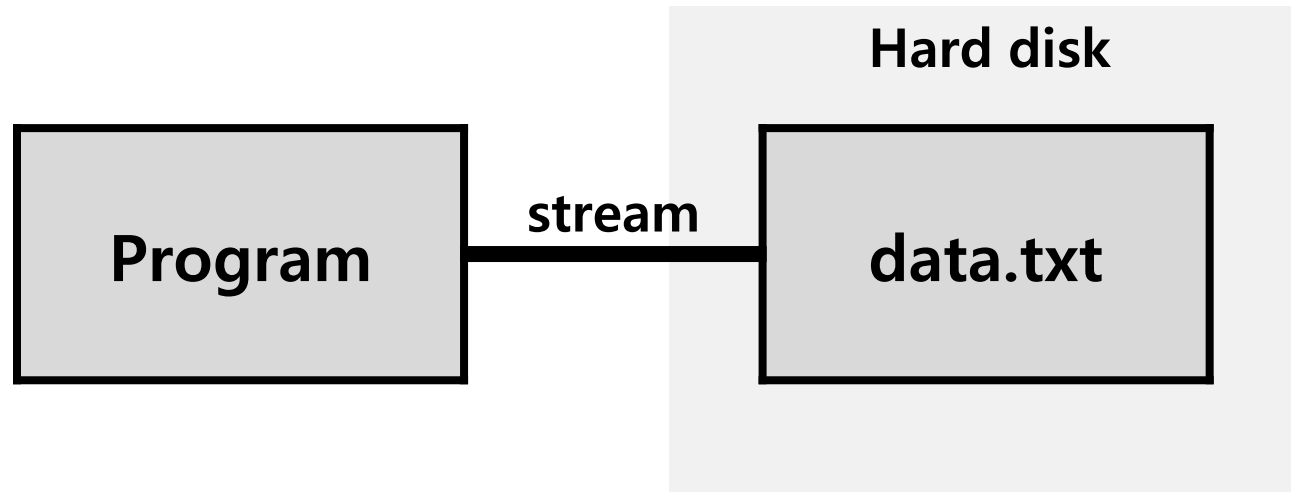
ROBIT
ROBOT SPORT GAME TEAM

파일 입출력

stream



ROBIT
ROBOT SPORT GAME TEAM



프로그램상에서 파일에 저장되어 있는 데이터를 참조하길 원할 때,
프로그램과 참조할 데이터가 저장되어 있는 파일 사이에
데이터가 이동할 수 있는 다리의 역할

→ stream을 형성해야 데이터를 주고 받을 수 있음

→ 파일 스트림 형성 예 : **FILE *** 구조체포인터

파일 입출력



ROBIT
ROBOT SPORT GAME TEAM

FILE * fopen(const char * filename, const char * mode);

- 스트림을 형성할 때 호출하는 함수
- 해당 파일과의 스트림을 형성하고 스트림 정보를 FILE구조체에 담아 그 변수의 주소 값 반환
- 실패 시 NULL 포인터 반환
- 첫 번째 인자(filename) : 스트림을 형성할 파일의 경로와 이름
- 두 번째 인자(mode) : 형성할 스트림의 종류(파일의 접근 모드와 파일 입출력 모드)

int fclose(FILE * stream)

- 스트림을 해제하는 함수
- 파일을 닫는 함수
- 성공 시 0 반환



파일 입출력 모드

- 스트림의 종류 지정

- 파일의 접근 모드

r	• 읽기(read) 전용으로 파일을 엽니다. • 파일이 없거나 찾을 수 없는 경우에 호출 실패
w	• 쓰기(write) 전용으로 파일을 엽니다. • 지정한 파일명이 있는 경우: 파일 내용을 모두 지우고 새로 만듭니다. • 지정한 파일명이 없는 경우: 새로운 파일을 생성 합니다.
a	• 추가(append) 쓰기 전용으로 파일을 엽니다. • 지정한 파일이 있으면 파일의 끝에서부터 내용을 추가합니다.
r+	• 파일을 읽고 쓰기 위해 엽니다. • 지정한 파일이 있는 경우: 기존의 내용을 덮어씁니다. • 지정한 파일이 없는 경우: 새로운 파일을 생성해서 데이터를 씁니다(저장).
w+	• 파일을 읽고 쓰기 위해 엽니다. • 지정한 파일이 있는 경우: 파일의 내용을 모두 지우고 새 파일을 만듭니다. • 지정한 파일이 없는 경우: 새로운 파일을 생성 한다.
a+	• 파일을 읽고 추가 쓰기 위해 엽니다. • 지정한 파일이 있으면 파일의 끝에서부터 내용을 추가합니다. • 나머지 기능은 r+와 같습니다.

- 파일의 입출력 모드

t	텍스트 파일 모드 (text file mode)
b	바이너리 파일 모드 (binary file mode)



파일 입출력 모드

- 스트림의 종류 지정

파일 접근
모드

r
w
a
r+
w+
a+

입출력 모드

+

t
b

=

파일 오픈 모드

rt	rb
wt	wb
at	ab
r+t	r+b
w+t	w+b
a+t	a+b

텍스트 모드의 경우 't' 생략 가능

```
FILE* stream;
```

```
stream=fopen("C:\\workspace\\data.txt", "r");
```

```
FILE* stream;
```

```
stream=fopen("C:\\workspace\\data.txt", "w");
```

```
FILE* stream;
```

```
stream=fopen("C:\\workspace\\data.txt", "rt");
```

```
FILE* stream;
```

```
stream=fopen("C:\\workspace\\data.txt", "wt");
```

```
FILE* stream;
```

```
stream=fopen("C:\\workspace\\data.txt", "ab");
```

파일 입출력

- 파일 쓰기

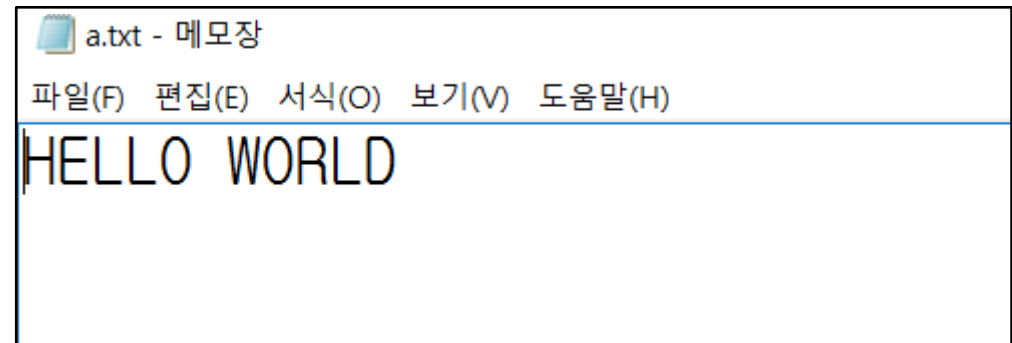
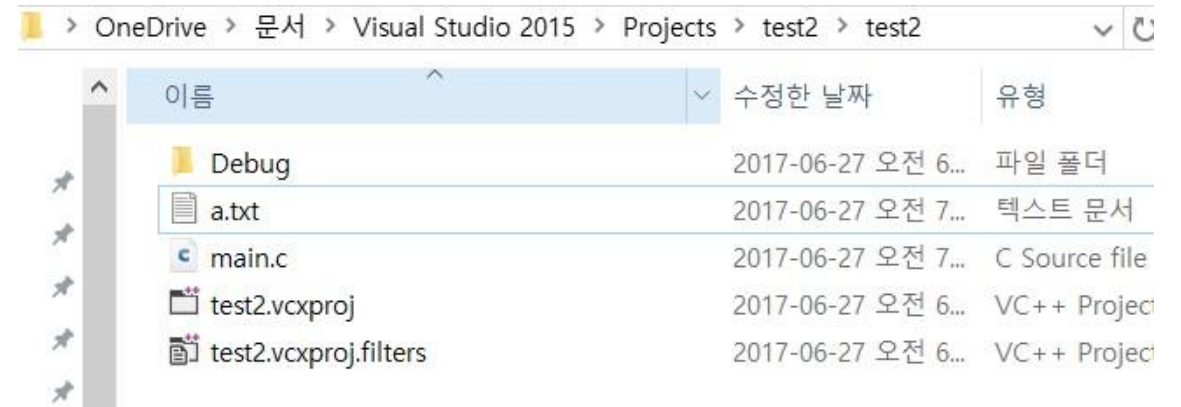
```
int main()
{
    FILE *F;

    F = fopen("a.txt", "w");
    fprintf(F, "HELLO WORLD");
    fclose(F);

    return 0;
}
```



RO:BIT
ROBOT SPORT GAME TEAM





파일 입출력

- 파일 읽기

```
#include <stdio.h>
|
int main()
{
    FILE *F;
    char string[15];
    char string2[15];

    F = fopen("a.txt","r");

    fscanf(F,"%s",string);
    fscanf(F,"%s",string2);

    printf("%s\n",string);
    printf("%s\n",string2);
    fclose(F);
    return 0;
}
```

```
HELLO
WORLD
계속하려면 아무 키나 누르십시오 . . .
```

파일에 출력하기



ROBIT
ROBOT SPORT GAME TEAM

```
#include <stdio.h>

int main() {
    FILE * fp = fopen("write_test.txt", "w");

    fputc('a', fp);
    fputs("name : 김땡땡", fp);
    fprintf(fp, "age : %d", 20);

    fclose(fp);
    return 0;
}
```



write_test.txt - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

a

name : 김땡땡

age : 20

파일 읽어 오기



ROBIT
ROBOT SPORT GAME TEAM

```
#include <stdio.h>

int main() {
    FILE * fp = fopen("write_test.txt", "r");

    char ch;
    char str1[255];
    char str2[255];

    ch = fgetc(fp);
    fgetc(fp);
    fgets(str1, 255, fp);
    fgets(str2, 255, fp);
    printf("%c\n%s%s\n", ch, str1, str2);

    fclose(fp);
    return 0;
}
```

```
a
name : 김땡땡
age : 20
계속하려면 아무 키나 누르십시오 . . .
```

- **fgetc**
: 개행 문자(\n) 만날 때 까지 읽어 옴.
- **fgets**
: 개행 문자나 공백(space bar)만날 때까지 읽어 옴.

참고



ROBIT
ROBOT SPORT GAME TEAM

파일 입출력 과정

step	과정	수행내용
1	파일 스트림 생성	파일 포인터 생성 FILE* stream;
2	파일 오픈	fopen()
3	파일 입출력 수행	fgetc(), fputc(), fgets(), fputs(), fscanf(), fprintf(), fread(), fwrite(), fseek(), ftell() 등 함수 사용
4	파일 닫기	fclose()

표준 파일 입출력 함수

- 대표적인 표준 파일 입출력 함수
- fgetc() 함수와 fputc() 함수
- fgets() 함수와 fputs() 함수
- fprintf() 함수와 fscanf() 함수
- feof() 함수
- fflush() 함수
- fread() 함수와 fwrite() 함수
- fseek() 함수와 ftell() 함수

int getc (void);	int fgetc (FILE* stream);	문자 단위 입 력	stdin
int putchar (int c);	int fputc (int c, FILE* stream);	문자 단위 출 력	stdout
char* gets (char *s);	char* fgets (char *s, int n, FILE* stream);	문자열 단위 입력	stdin
int puts (char* str);	int fputs (const char* s, FILE* stream);	문자열 단위 출력	stdout



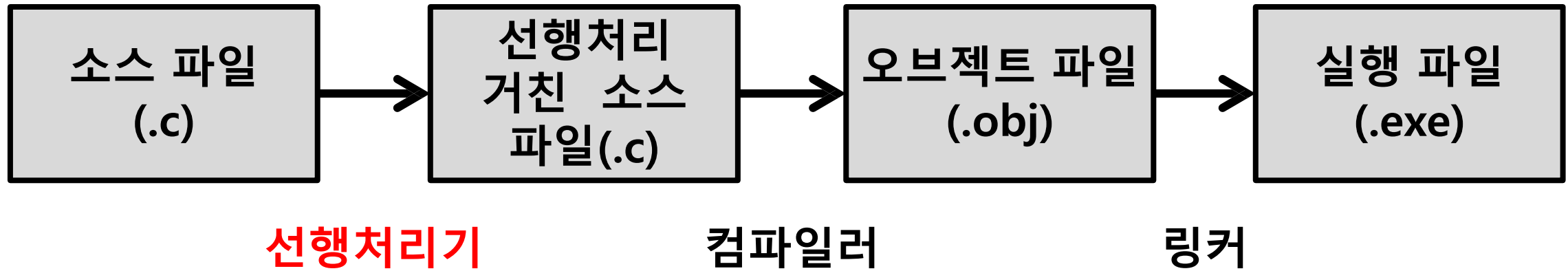
ROBIT
ROBOT SPORT GAME TEAM

선행 처리

실행파일 생성 과정



ROBIT
ROBOT SPORT GAME TEAM



- 선행 처리 : 컴파일 이전의 처리
- 선행 처리의 과정을 거쳐서 생성되는 파일 : 소스 파일
- 선행처리기 : 삽입해 놓은 선행처리 명령문대로 소스코드의 일부를 수정

선행 처리 예시



ROBIT
ROBOT SPORT GAME TEAM

- 선행처리 명령문은 #문자로 시작
- 컴파일러가 아닌 선행처리기에 의해서 처리되는 문장이기 때문에 끝에 세미콜론을 붙이지 않는다.

`#define SIZE 2` = SIZE를 만나면 2로 치환해라!!

```
typedef struct _Student
{
    int number;
    char name[10];
    double grade;
}Student;

void main(void)
{
    Student list[SIZE]
    int i = 0;

    for(i = 0; i < SIZE; i++)
    {
        printf("학번을 입력하시오: ");
        scanf("%d", &list[i].number);
        printf("이름을 입력하시오: ");
        scanf("%s", list[i].name);
        printf("학점을 입력하시오(실수): ");
        scanf("%lf", &list[i].grade);
    }

    for(i = 0; i < SIZE; i++)
    {
        printf("학번: %d, 이름: %s, 학점: %f\n", list[i].number, list[i].name, list[i].grade);
    }
}
```

선행처리기



명령문 소멸

```
typedef struct _Student
{
    int number;
    char name[10];
    double grade;
}Student;

void main(void)
{
    Student list[ 2 ];
    int i = 0;

    for(i = 0; i < 2 ; i++)
    {
        printf("학번을 입력하시오: ");
        scanf("%d", &list[i].number);
        printf("이름을 입력하시오: ");
        scanf("%s", list[i].name);
        printf("학점을 입력하시오(실수): ");
        scanf("%lf", &list[i].grade);
    }

    for(i = 0; i < 2 ; i++)
    {
        printf("학번: %d, 이름: %s, 학점: %f\n", list[i].number, list[i].name, list[i].grade);
    }
}
```

대표적인 선행처리 명령문



ROBIT
ROBOT SPORT GAME TEAM

① define : Object-like macro

#define

지시자

PI

매크로

3.141592

매크로 몸체

매크로 PI를 매크로 몸체 3.141592fh 전부 치환해라



PI라는 이름의 매크로 = 상수 3.141592

“매크로 상수”



대표적인 선행처리 명령문

① define : Object-like macro

```
#include <stdio.h>

#define NAME      "김땡땡"
#define AGE       20
#define PRINT_ADDR printf("주소 : 서울시%Wn");

int main() {
    printf("이름 : %s Wn", NAME);
    printf("나이 : %d Wn", AGE);
    PRINT_ADDR;

    return 0;
}
```

```
이름 : 김땡땡
나이 : 20
주소 : 서울시
계속하려면 아무 키나 누르십시오 . . .
```

대표적인 선행처리 명령문



ROBIT
ROBOT SPORT GAME TEAM

② define : Function-like macro

#define

지시자

SQUARE(X)

매크로 함수

X*X

매크로 몸체

매개변수 존재



함수와 유사하게 동작

“매크로 함수”

대표적인 선행처리 명령문



ROBIT
ROBOT SPORT GAME TEAM

② define : Function-like macro

```
#include <stdio.h>

#define SQUARE(X) X*X

int main() {
    int num = 5;

    printf( "%d*%d = %d\n", num, num, SQUARE(num) );
    printf( "%d*%d = %d\n", 2+3, 2+3, SQUARE(2+3) );

    return 0;
}
```



대표적인 선행처리 명령문

② define : Function-like macro

```
#include <stdio.h>
```

```
#define SQUARE(X) X*X
```

```
int main() {
```

```
    int num = 5;
```

```
    printf("%d*d = %d\n", num, num, SQUARE(num));
```

```
    printf("%d*d = %d\n", 2+3, 2+3, SQUARE(2+3));
```

```
    return 0;
```

```
}
```

5*5 = 25

5*5 = 11

계속하려면 아무 키나 누르십시오 . . .

선행 처리기는
단순 치환을 한다.

→ $X=2+3$

$X*X = 2+3*2+3 = 11$

※ $SQUARE((2+3))$ 의 경우

→ $X=(2+3)$

$X*X = (2+3)*(2+3) = 25$



대표적인 선행처리 명령문

② define : Function-like macro

```
#include <stdio.h>
```

```
#define SQUARE(X) (X)*(X)
```

```
int main() {
```

```
    int num = 5;
```

```
    printf("%d*d = %d\n", num, num, SQUARE(num));
```

```
    printf("%d*d = %d\n", 2+3, 2+3, SQUARE(2+3));
```

```
    return 0;
```

```
}
```

5*5 = 25

5*5 = 25

계속하려면 아무 키나 누르십시오 . . .



대표적인 선행처리 명령문

- **함수 호출 :**
호출된 함수를 위해 스택 메모리를 할당하고, 실행위치로 이동 후 매개변수로 인자를 전달하고, 이후 return문에서 값을 반환함.
- **매크로 함수:**
선행처리기가 매크로 함수의 호출 문장을 매크로 함수의 몸체부분으로 치환함.

[장점]

- 일반 함수에 비해 실행속도 빠름
- 인수로 어떤 자료형의 변수든 입력 가능.

[단점]

- 정의하기 까다로움
- 디버깅하기 쉽지 않음

→ 간단하고, 호출 빈도수가 높은 함수를 매크로 함수로 정의.



대표적인 선행처리 명령문

③ 조건부 컴파일을 위한 매크로

(1) #if ~ #endif If 조건이 참인 블록만 실행됨!

- 두 지시자 사이에 존재하는 코드는 조건에 따라 삽입 및 삭제됨.
- #if문의 뒤에는 반드시 #endif문이 와야 함.

```
#include <stdio.h>

#define ADD 1
#define MIN 0

int main() {
    int num1, num2;
    printf("정수 2개 입력 : ");
    scanf("%d %d", &num1, &num2);

    #if ADD //ADD 가 참이라면
        printf("%d + %d = %dWn", num1, num2, num1 + num2);
    #endif

    #if MIN //MIN 이 참이라면
        printf("%d - %d = %dWn", num1, num2, num1 - num2);
    #endif

    return 0;
}
```

정수 2개 입력 : 1 2
1 + 2 = 3
계속하려면 아무 키나 누르십시오 . . .

```
#include <stdio.h>
#define NUM 0

int main() {
    #if NUM == 0
        printf("num = 0Wn");
    #elif NUM == 1
        printf("num = 1Wn");
    #elif NUM == 2
        printf("num = 2Wn");
    #elif NUM == 3
        printf("num = 3Wn");
    #else
        printf("num >= 4");
    #endif

    return 0;
}
```

num = 0
계속하려면 아무 키나 누르십시오 . . .



대표적인 선행처리 명령문

③ 조건부 컴파일을 위한 매크로

(2) `#ifdef ~ #endif` 정의되어 있는 블록만 실행됨!

(3) `#ifndef ~ #endif` 정의되어 있지 않은 블록만 실행됨!

- 정의되어있는 값에 상관없이 정의되었냐, 정의되지 않았느냐를 기준으로 동작함.
- 정의 되어있을 때를 참값으로 함.

```
#include <stdio.h>

//#define ADD 1
#define MIN 0

int main() {
    int num1, num2;
    printf("정수 2개 입력 : ");
    scanf("%d %d", &num1, &num2);

#ifdef ADD //ADD 가 정의되었다면
    printf("%d + %d = %d\n", num1, num2, num1 + num2);
#endif

#ifdef MIN //MIN 이 정의되었다면
    printf("%d - %d = %d\n", num1, num2, num1 - num2);
#endif

    return 0;
}
```

정수 2개 입력 : 1 2
1 - 2 = -1
계속하려면 아무 키나 누르십시오 . . .



ROBIT
ROBOT SPORT GAME TEAM

과제

과제1

단순 연결 리스트 구현



ROBIT
ROBOT SPORT GAME TEAM

- insert : 원하는 위치에 node 추가(그 전 data와 index모두 가능하도록)
- insert_back : 연결리스트의 맨 끝에 node 추가
- insert_first : 연결리스트의 맨 처음에 node 추가
- delete : 원하는 요소 삭제(원하는 요소는 data와 index모두 가능하도록)
- delete_first : 연결리스트 맨 처음 node 삭제
- delete_back : 연결리스트 맨 마지막 node 삭제
- get_entry : 요소 찾기(data로 찾을 시 index 반환, index로 찾을 시 data 반환)
- get_length : 리스트 전체 길이 반환
- print_list : 리스트의 모든 요소 출력
- reverse : 리스트 역순으로 만들기
- Data 자료형은 자유

과제2

단순 연결 리스트를 이용한 stack구현

- push : 정수 push
- pop : pop하고 pop된 값 출력. stack이 비어있을 시 비어있다고 출력
- size : stack 크기 출력
- top : top에 위치한 값 반환.
- isEmpty : stack에 데이터가 없으면 true, 있으면 false 반환
- printStack : stack 내 모든 값 출력. stack이 비어있을 시 비어있다고 출력
- Data 자료형 자유



ROBIT
ROBOT SPORT GAME TEAM

```
typedef struct _Node {  
    int data;  
    struct _Node * next;  
}Node;  
  
typedef struct Stack {  
    Node * top;  
    int size;  
};|
```

과제3

단순 연결 리스트를 이용한 Queue 구현



ROBIT
ROBOT SPORT GAME TEAM

- Enqueue : Queue에 data 입력
- Dequeue : Dequeue하고 Dequeue된 값 출력. Queue가 비어있을 시 비어있다고 출력
- size : Queue 크기 출력
- front : front에 위치한 값 반환.
- rear : rear에 위치한 값 반환.
- isEmpty : Queue에 데이터가 없으면 true, 있으면 false 반환
- printQueue : Queue 내 모든 값 출력. Queue가 비어있을 시 비어있다고 출력

과제4

회문 판별하기



ROBIT
ROBOT SPORT GAME TEAM

- 회문 또는 팰린드롬(Palindraome)은 앞 뒤 방향으로 볼 때 같은 순서의 문자로 구성된 문자열을 말한다.
- Ex) abba, madam, was it a cat i saw
- 문자열을 입력받아 회문인지 아닌지 판별하여 출력
- hint :
- 스택과 큐를 이용(LIFO, FIFO)

과제 5



ROBIT
ROBOT SPORT GAME TEAM

출석부 프로그램

- Student 구조체 : 번호, 이름, 주소(format : 나라,도,시,구), 성적
- 구조체 배열, 구조체 포인터를 이용한 함수 이용
- 모든 항목 예외처리(ex 숫자, 문자)
- 필수 기능)
 1. 학생 정렬 기능
: 번호순, 이름순, 주소순(나라,도,시,구에 따라 따로 진행), 성적순 출력 모두 가능
 2. 학생 찾기 기능
: 번호or주소(나라,도,시,구에 따라 따로 진행)or성적을 입력하면 해당하는 모든 학생 이름 출력
 3. 학생 추가, 삭제 기능
: 번호, 이름, 주소, 성적 입력하면 출석부에서 해당 학생 추가/삭제. 중복된 경우 선택하여 삭제
 4. 출석부 저장 및 불러오기
: 파일 입출력을 이용해 출석부를 저장하고 불러 오